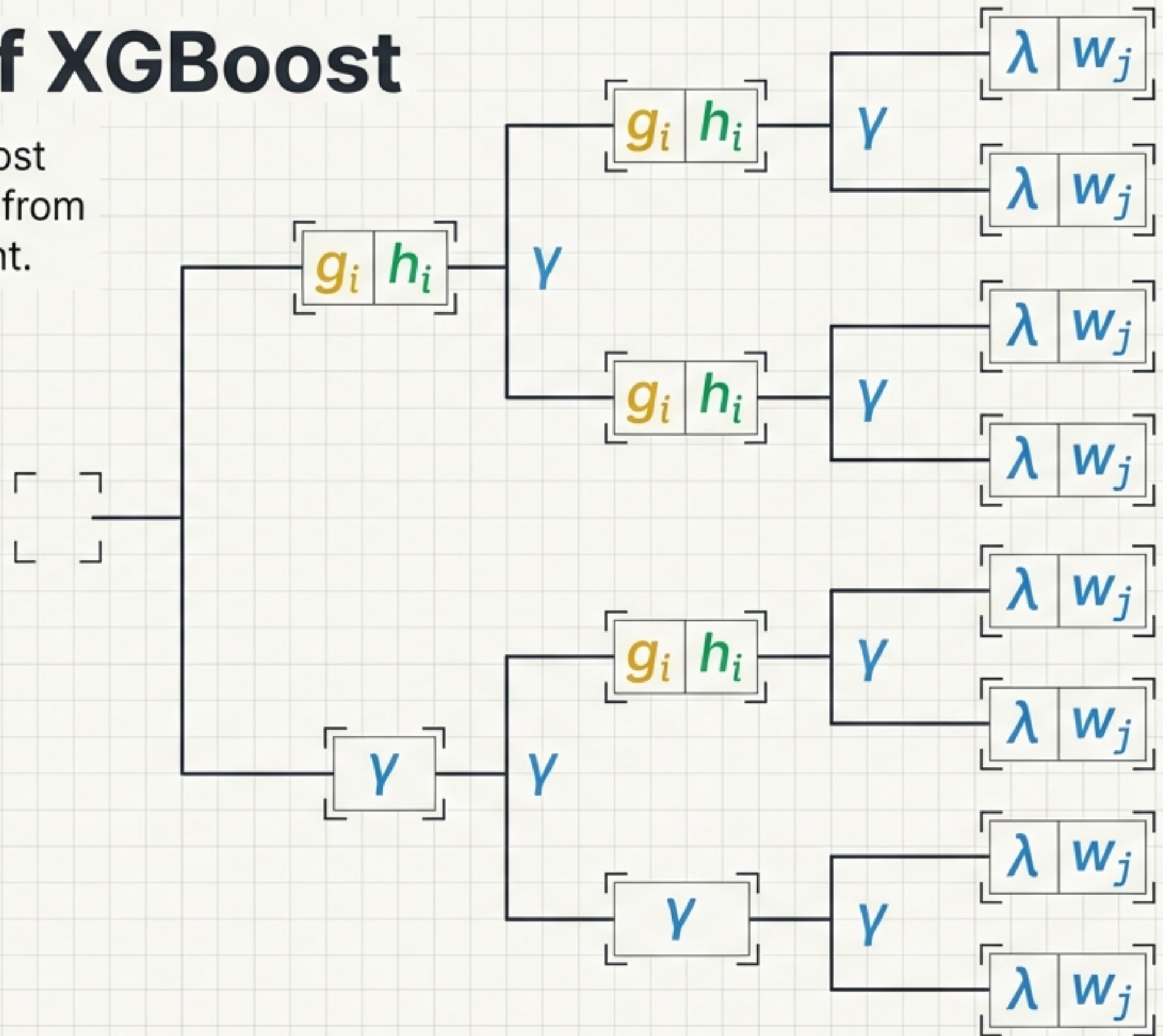
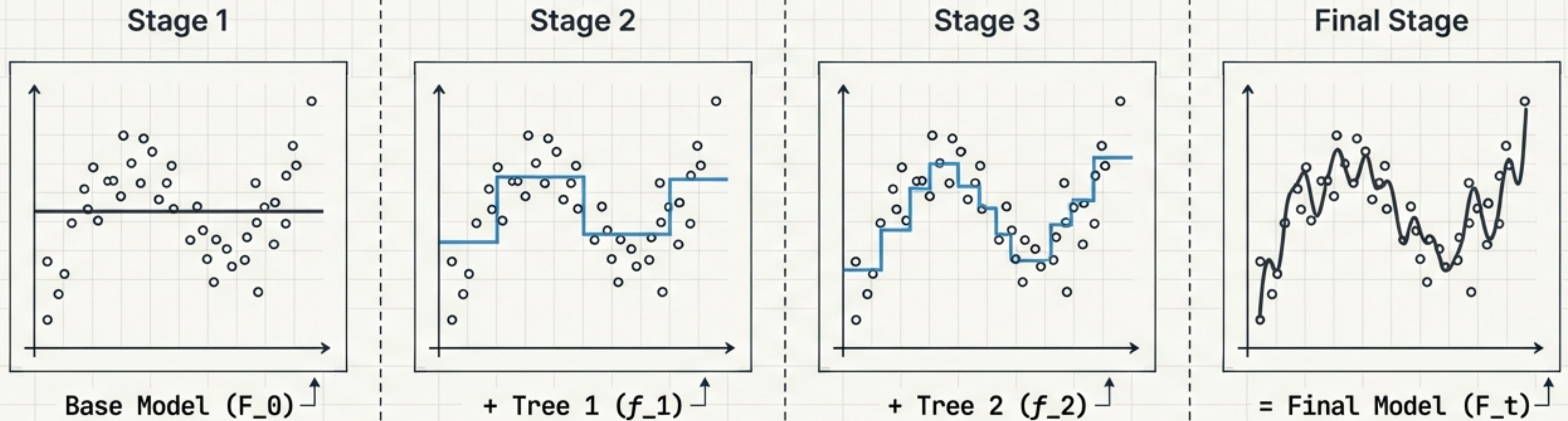


The Mathematics of XGBoost

A visual derivation of machine learning's most powerful algorithm—tracing the exact path from the objective function to the final leaf weight.



[The Additive Modelling Foundation]



$$\left[F_t(x_i) = F_{t-1}(x_i) + f_t(x_i) \right]$$

Gradient boosting builds models in a stage-wise fashion. At any step t , the combined predictions of all previous models (F_{t-1}) are fixed. Our sole mathematical objective is to find the single best new tree (f_t) to correct the remaining errors.

The single new tree we must optimise at step t .

Defining Success: The Objective Function

To find the optimal new tree, we must minimise this exact objective function. It explicitly balances predictive accuracy against structural simplicity.

$$Obj = \sum_{i=1}^n L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

The Loss Function

Pushing the model to fit the data perfectly.
Measures the gap between actuals (y_i) and predictions ($\hat{y}_i$).

The Regularisation Term

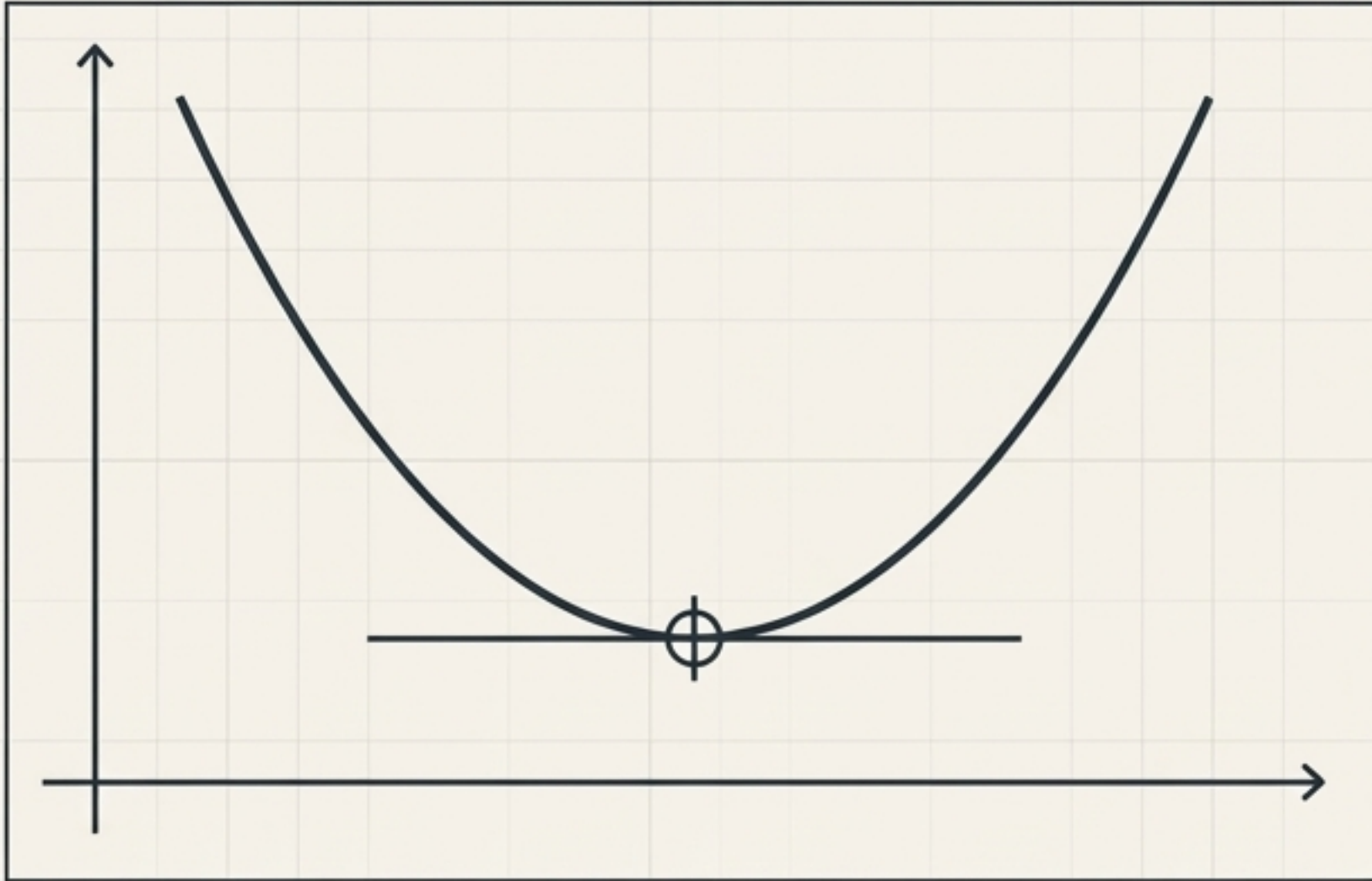
Punishing the model for complexity.
Prevents overfitting.

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

T = number of leaves
 w_j = leaf weights

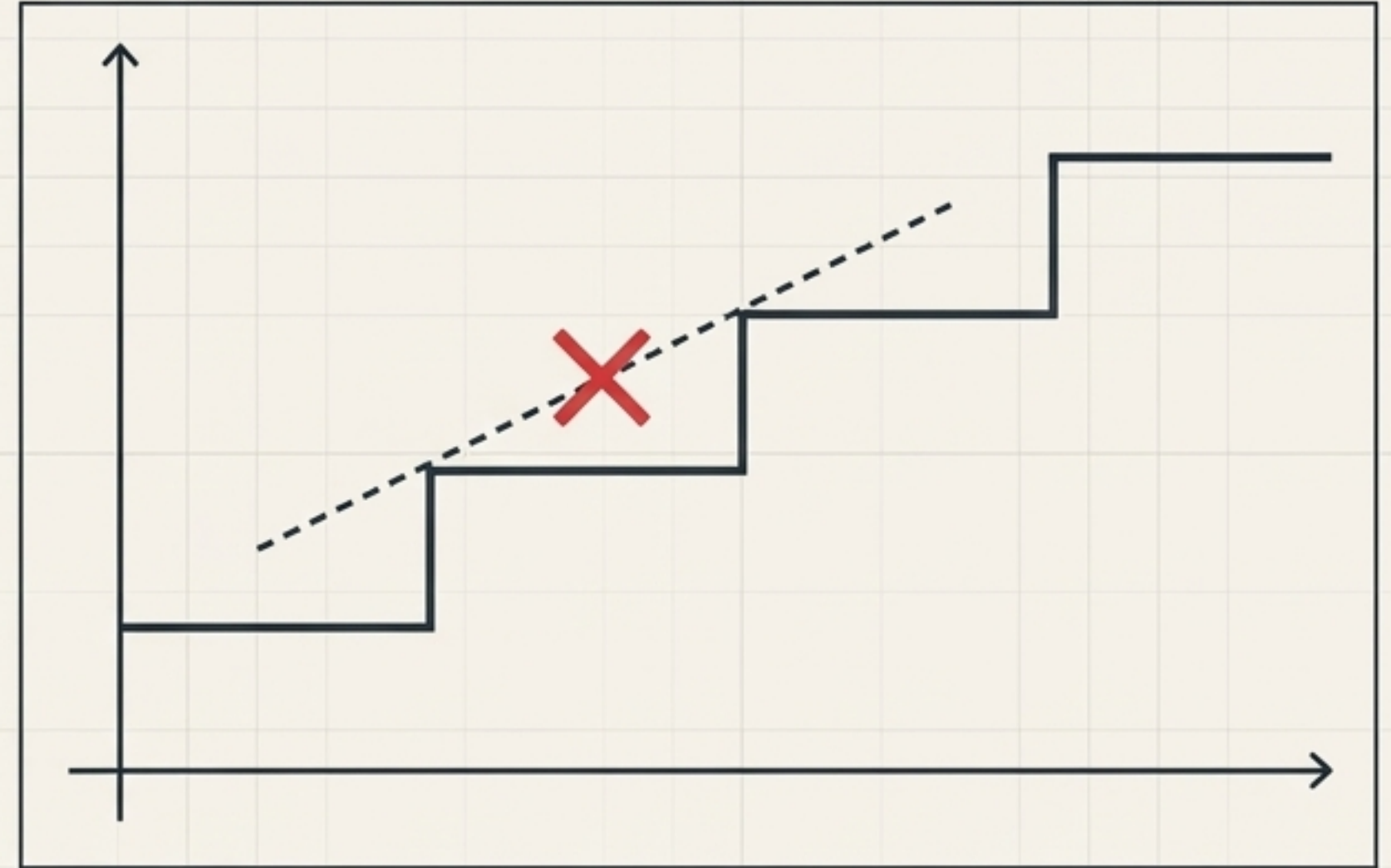
[The Mathematical Roadblock of Decision Trees]

Linear Regression



Smooth & Differentiable. Easy to find the minimum.

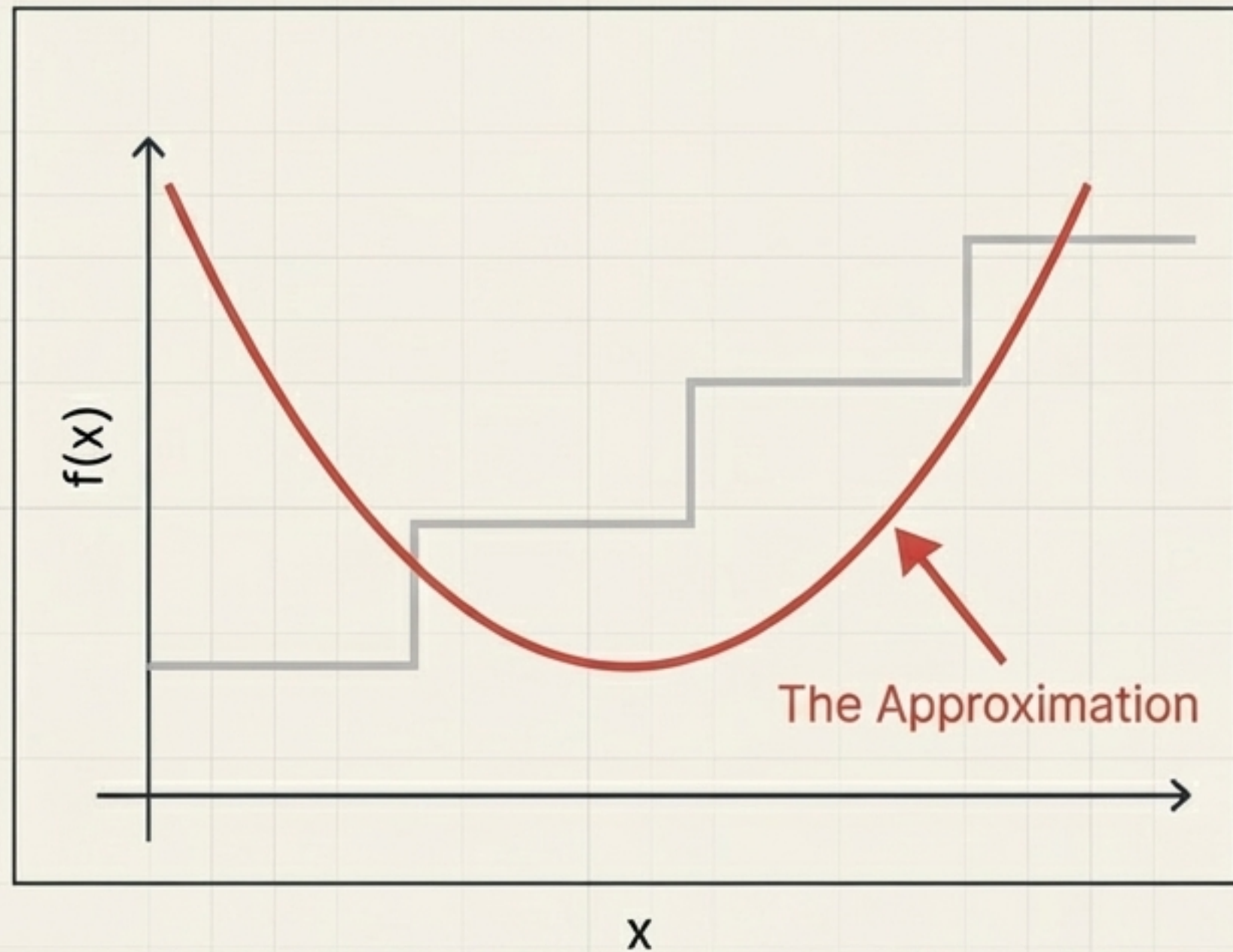
Decision Trees



Discontinuous & Non-Differentiable. Calculus fails here.

To minimise a function, we typically calculate its derivative and set it to zero. However, because decision trees output discrete, stepped values, our loss curve becomes jagged. Standard gradient descent algorithms instantly fail here.

[The Hero Technique: Second-Order Taylor Approximation]



The Equation

$$f(x) \approx f(a) + f'(a)(x-a) + \frac{1}{2}f''(a)(x-a)^2$$

The Gradient (g_i)

The First Derivative.
Represents the slope of the curve.

The Hessian (h_i)

The Second Derivative.
Represents the curvature of the parabola.

If we cannot differentiate the exact function, we approximate it. By applying a second-order Taylor Expansion, we transform our jagged roadblock into a smooth, perfectly differentiable quadratic curve.

[Rewriting the Objective Function]

$$\text{Obj} = \sum_{i=1}^n \left[\cancel{L(y_i, \hat{y}^{(t-1)})} + \overset{\substack{\text{Constant from previous iterations.} \\ \text{Drops out of optimisation.}}}{g_i} f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

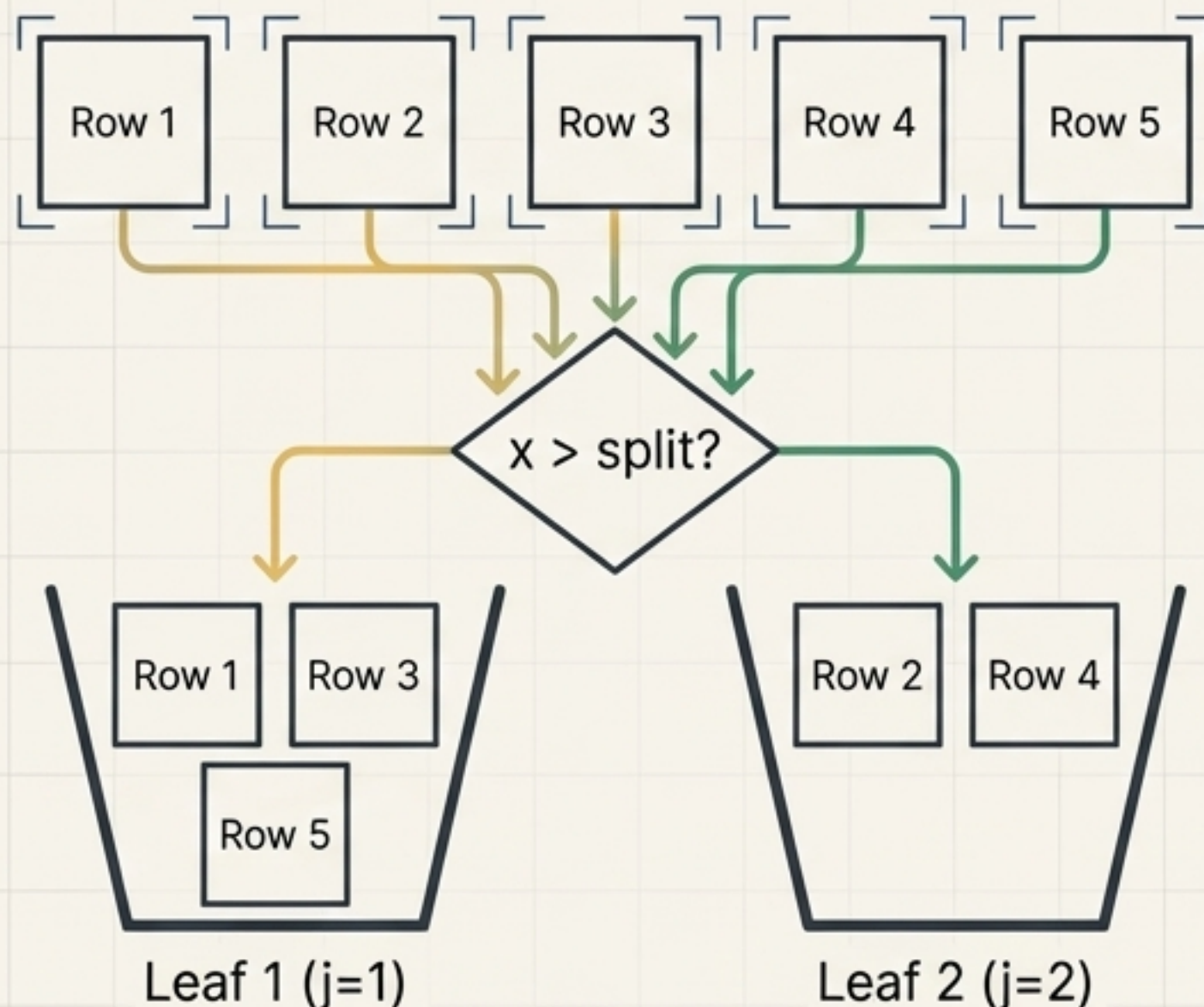


$$\text{Obj} = \sum_{i=1}^n \left[\underset{\substack{\text{Gradient multiplied} \\ \text{by the new tree.}}}{g_i} f_t(x_i) + \frac{1}{2} \underset{\substack{\text{Half the Hessian} \\ \text{multiplied by the} \\ \text{new tree squared.}}}{h_i} f_t^2(x_i) \right] + \Omega(f_t)$$

By applying the Taylor approximation and **dropping constants** from previous iterations, we distil the objective function into a pure **quadratic optimisation problem** focused entirely on the new tree, $f_t(x_i)$.

The Great Reorganisation: From Instances to Leaves

The Physical Funnel



The Equation Swap

~~$$\sum_{i=1}^n \text{Loss}$$~~ Iterating through all rows.

→

$$\sum_{j=1}^T \sum_{i \in I_j}$$

Iterating through all leaves. Only the rows inside that specific leaf.

Instead of calculating loss row-by-row, XGBoost regroups the math leaf-by-leaf. A critical conceptual pivot that allows us to calculate the collective gradient and hessian for all data points landing in the same bucket.

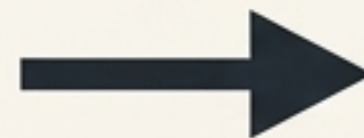
[Defining the Optimal Leaf Weight Variable]

$$\text{Obj} = \sum_{j=1}^T \left[\underbrace{\left(\sum_{i \in I_j} g_i \right)}_{\text{Sum of Gradients in leaf } j} w_j + \underbrace{\frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right)}_{\text{Sum of Hessians} + \text{Regularisation penalty}} w_j^2 \right] + \gamma T$$

Sum of Gradients in leaf j

Sum of Hessians + Regularisation penalty

$$f_t(x_i)$$



$$w_j$$

Because all instances in a specific leaf share the exact same prediction, the tree function becomes a single constant weight.

[By viewing the tree leaf-by-leaf, we replace the complex tree function with a standalone variable: w_j .
Our entire objective function is now a clean, independent quadratic equation for **every single leaf**.]

[Optimising for the Perfect Leaf Weight]

1. Take derivative with respect to w_j

$$\frac{\partial \theta_j}{\partial w_j} = \sum g_i + (\sum h_i + \lambda) w_j$$

2. Set derivative to 0 to find the minimum slope

$$0 = \sum g_i + (\sum h_i + \lambda) w_j$$

3. Solve for the optimal weight (w_j^*)

$$w_j^* = - \frac{\sum g_i}{\sum h_i + \lambda}$$

Because we successfully transformed the loss curve into a simple parabola, we can easily find the exact bottom using standard calculus. This single master formula determines the optimal output value for every leaf in an XGBoost tree.

[The Master Formula Matrix: Regression vs. Classification]

	Regression (Mean Squared Error)	Classification (Log Loss)
The Math Definitions	$\text{Gradient } (g_i) = \text{Residuals}$ $\text{Hessian } (h_i) = 1$	$\text{Gradient } (g_i) = \text{Residuals}$ $\text{Hessian } (h_i) = p(1-p)$
The Resulting Leaf Weight (w_j^*)	$w_j^* = \frac{\text{Sum of Residuals}}{\text{Count} + \lambda}$	$w_j^* = \frac{\text{Sum of Residuals}}{\sum [p(1-p)] + \lambda}$

The brilliance of XGBoost lies in its abstraction. Under the hood, Regression and Classification are the exact same mathematical problem. Only the specific definitions of the first and second derivatives change.

[Deriving the Similarity Score]

$$Obj = (\sum g_i) w_j + \frac{1}{2} (\sum h_i + \lambda) w_j^2$$

$$w_j^* = - \frac{\sum g_i}{\sum h_i + \lambda}$$

$$Obj = (\sum g_i) \left[- \frac{\sum g_i}{\sum h_i + \lambda} \right] + \frac{1}{2} (\sum h_i + \lambda) \left[- \frac{\sum g_i}{\sum h_i + \lambda} \right]^2$$

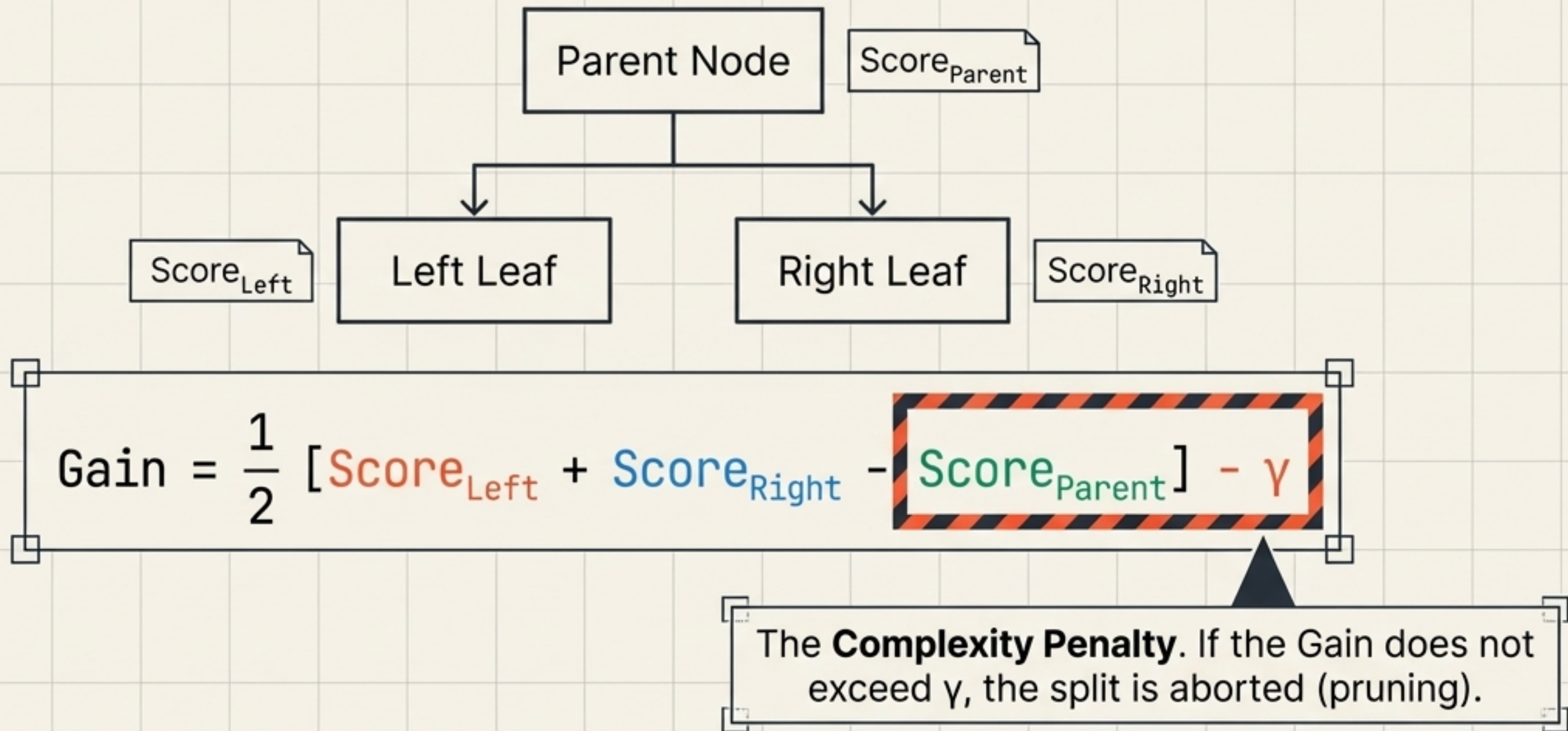
$$Obj = - \frac{(\sum g_i)^2}{\sum h_i + \lambda} + \frac{1}{2} \frac{(\sum g_i)^2}{\sum h_i + \lambda}$$

$$\text{Score} = \frac{(\sum g_i)^2}{\sum h_i + \lambda}$$

If we know the perfect leaf weight, we can calculate exactly how much loss that leaf will eliminate. This 'Similarity Score' allows us to measure the mathematical quality of any proposed tree structure.

This replaces Gini Impurity and Entropy. It represents the maximum possible loss reduction achievable by this specific group of data points.

[The Split Criteria: Evaluating Information Gain]



To build the tree, XGBoost scans potential splits and calculates the Gain. By subtracting the Parent's score from the new Children's scores, it calculates the precise loss reduction. The γ parameter enforces a strict mathematical threshold to prevent overfitting.

[The Complete Mathematical Engine]

[Calculate Gradients & Hessians]

Evaluate g_i and h_i from previous predictions

[Evaluate Splits]

Use the Gain formula to find optimal structure

[Calculate Leaf Weights]

Apply w_j^* formula to finalize outputs

[Update Predictions]

$$F_t = F_{t-1} + f_t$$

Loop

Return to Step 1 until loss is minimized

The sheer power of XGBoost comes from running this rigorous calculus iteratively. Every single tree is mathematically guaranteed to be the optimal structure to fix the precise errors of the trees that came before it.

[Why XGBoost Dominates Tabular Data]

By translating algorithmic heuristics into pure, differentiable **calculus**, XGBoost ensures that every split, weight, and tree is the **mathematically optimal choice**.



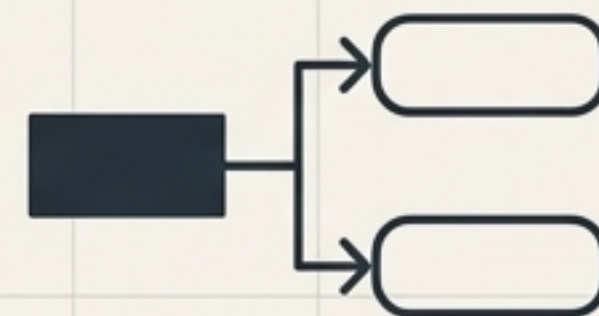
[Built-in Regularisation]

Unlike standard gradient boosting, it integrates **complexity penalties** (λ , γ) directly into the root root mathematical objective, systematically preventing overfitting.



[Taylor Approximation]

By utilizing **2nd-order derivatives**, it effortlessly bypasses non-differentiable roadblocks, optimizing highly complex, custom **loss functions**.



[Unified Framework]

It abstracts the learning process beautifully. **Regression** and **Classification** are solved are solved using the exact same underlying mathematical architecture.